Review

39/60 correct · 21 wrong

All (60) | Wrong (21) | Flagged (0) | Customer Support | Claude Code | Code Generation | Multi-Agent Research

Q1 D1 Multi-Agent Research System CORRECT

Your coordinator agent is configured correctly and its system prompt instructs it to spawn subagents using the Task tool. In production, every coordinator run completes in a single turn without any subagents starting — the coordinator produces a direct response instead of delegating. No errors appear in the logs. What is the most likely configuration error?

A The coordinator's `allowedTools` list does not include `"Task"`

B The coordinator's system prompt does not explicitly name the Task tool by its identifier.

C The subagent `AgentDefinition` objects are missing a required `"parentId"` field linking them to the coordinator.

D The coordinator is calling the wrong API endpoint.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Task Statement 1.3 specifies that `allowedTools` must include `"Task"` for a coordinator to invoke subagents. Without this, the coordinator lacks the capability to spawn child agents regardless of its system prompt instructions.

OPTION B

The fundamental issue is the capability permission, not prompt framing — without `"Task"` in `allowedTools`, no amount of instruction enables spawning.

OPTION C

No `parentId` field is required in `AgentDefinition`; the Task tool handles the delegation relationship.

OPTION D

Subagent spawning uses the standard Claude API; there is no separate `/v1/agents` endpoint.

PRACTICE TAKEAWAY

Guiding principle: *Tool authorization should be enforced by configuration, not by prompt instructions.*
Design guidance: *Use* `allowedTools` *to grant only the capabilities required for the current role or environment, denying future tools by default.*

Review topic: Agentic Architecture

Q2 D1 Multi-Agent Research System CORRECT

You are implementing an agentic loop for a research coordinator. The coordinator calls web search and document analysis tools iteratively until it has gathered sufficient information, then returns a final cited report. Monitoring shows the loop sometimes never terminates — it keeps making tool calls even after gathering complete information. What is the correct loop termination condition?

A Terminate when the response body contains no tool call XML tags.

B Terminate when the coordinator's response includes a phrase like "I have completed my research"

C Terminate after a fixed maximum number of iterations with a configurable cap.

D Terminate when `stop_reason` equals `"end_turn"`; continue the loop by executing the requested tool when `stop_reason` equals `"tool_use"`.

WHY THE OTHER CHOICES MISS

OPTION A

Parsing natural language text for tool tags is an anti-pattern; the structured `stop_reason` field is the correct signal.

OPTION B

Natural language completion signals have a non-zero failure rate and are explicitly documented as an anti-pattern.

OPTION C

Arbitrary iteration caps are listed as an anti-pattern when used as the primary stopping mechanism.

CORRECT RATIONALE (OPTION D)

Task Statement 1.1 specifies implementing agentic loop control flow that continues when `stop_reason` is `"tool_use"` and terminates when `stop_reason` is `"end_turn"`. Parsing

`stop_reason` is the documented, reliable mechanism.

PRACTICE TAKEAWAY

Guiding principle: *Agentic loops should follow structured stop signals, not natural-language completion guesses or fixed caps.*
Design guidance: *Continue on* `stop_reason: "tool_use"`*, execute requested tools, append results, and terminate only on* `stop_reason: "end_turn"`*.*

Review topic: Agentic Architecture

Q3 D2 Multi-Agent Research System CORRECT

Your web search subagent returns an empty result array after querying a niche academic database for papers on "post-quantum cryptography in IoT edge devices." The coordinator cannot determine whether the empty result means the topic has no matching papers in that database or the database API timed out at 30 seconds due to a complex query. How should the search tool differentiate these cases?

A Return `{"results": [], "status": "empty"}` for genuine no-match results and `{"results": [], "status": "failed"}` for timeouts.

B Return `{"results": [], "isError": false}` for genuine empty results and `{"isError": true, "errorCategory": "transient", "isRetryable": true}` for timeout failures.

C Return an empty array in both cases and implement coordinator-side retry logic that automatically re-queries any search returning zero results up to three times before accepting the result as genuinely empty.

D Log the timeout to a monitoring service and return an empty array regardless.

WHY THE OTHER CHOICES MISS

OPTION A

A `status` string conveys similar information but lacks the standardised `errorCategory` and `isRetryable` fields that the MCP error pattern specifies.

CORRECT RATIONALE (OPTION B)

Task Statements 2.2 and 5.3 both address the distinction between access failures (requiring retry decisions) and valid empty results (representing successful queries with no matches). Structured error metadata with `errorCategory: "transient"` and

`isRetryable: true` for timeouts, versus a plain success response for genuine empty results, gives the coordinator the information it needs.

OPTION C

Retrying genuine empty results wastes resources and produces incorrect signal.

OPTION D

Hiding failures from the coordinator prevents recovery decisions.

PRACTICE TAKEAWAY

Guiding principle: *Tools should expose enough structured error information for the agent to choose retry, recovery, or escalation.*
Design guidance: *Return fields such as* `isError`, `errorCategory`, `isRetryable`, *and recovery metadata instead of relying on vague strings.*

Review topic: Tool Design & MCP

Q4 D3 Multi-Agent Research System CORRECT

Your research coordinator CLAUDE.md defines the default research methodology: source diversity requirements, citation format, and minimum evidence thresholds. A new enterprise client requires stricter source reliability thresholds (peer-reviewed only, no grey literature) and a different citation format (APA 7th edition instead of Chicago) for their commissioned reports only. All other research work should continue using the default configuration. What is the correct configuration approach?

A Add the client-specific rules to the root coordinator CLAUDE.md with a conditional block: "If the research brief contains CLIENT_ID, apply strict peer-review thresholds and APA 7th citation format."

B Create a client-specific subdirectory with its own CLAUDE.md that adds the peer-review thresholds and APA citation overrides, and run all client research jobs from that subdirectory.

C Create a `client-override.md` file and pass it at runtime using `--append-config client-override.md` for each commissioned client research run.

D Pass the client-specific citation format and source reliability requirements directly in the research brief user message at the start of each commissioned run.

WHY THE OTHER CHOICES MISS

OPTION A

CLAUDE.md does not support conditional logic — it is a static configuration document.

CORRECT RATIONALE (OPTION B)

CLAUDE.md files are inherited hierarchically — a subdirectory CLAUDE.md extends the parent without modifying it. Running client research from the client-specific subdirectory applies the override automatically while leaving default behavior intact for all other runs.

OPTION C

`--append-config` is not a documented Claude Code CLI flag.

OPTION D

User message instructions are soft constraints that do not provide the configuration-layer enforcement of CLAUDE.md hierarchical scoping.

PRACTICE TAKEAWAY

Guiding principle: *Claude Code behavior depends on where configuration is scoped and where the command is invoked.*
Design guidance: *Use project, directory, or user-level CLAUDE.md intentionally, and run Claude from the directory whose configuration should apply.*

Review topic: Claude Code Config

Q5 D2 Multi-Agent Research System CORRECT

Your research system has two tools: `analyze_content` (description: "Analyzes content") and `analyze_document` (description: "Analyzes documents"). Log analysis shows the coordinator routes document analysis tasks to `analyze_content` in 61% of cases and routes general content analysis tasks to `analyze_document` in 38% of cases — near-random selection between the two. What is the root cause and fix?

A The tools have identical parameter signatures, causing the SDK to route calls between them randomly when it cannot distinguish which tool the coordinator intended.

B The coordinator's system prompt does not reference either tool by name.

C The model's context window is too small to hold both tool descriptions simultaneously and attend to them together, causing degraded selection between the two similar tools.

D The near-identical one-line descriptions prevent the coordinator from differentiating them.

WHY THE OTHER CHOICES MISS

OPTION A

Parameter signatures affect how tools are called, not which tool is selected; selection happens based on descriptions.

OPTION B

Mentioning tool names in the system prompt adds noise without clarifying differentiation.

OPTION C

Context window size is not the issue; both short descriptions fit easily — the issue is description quality, not length.

CORRECT RATIONALE (OPTION D)

Task Statement 2.1 identifies minimal and overlapping tool descriptions as the root cause of misrouting between similar tools. Tool descriptions are the primary mechanism LLMs use for tool selection; expanding them to clarify purpose, input formats, and differentiation boundaries directly addresses the root cause.

PRACTICE TAKEAWAY

Guiding principle: *Tool descriptions are the primary selection signal; overlapping descriptions cause near-random routing.*
Design guidance: *Expand each tool's description with purpose, input formats, and explicit boundaries versus similar tools.*

Review topic: Tool Design & MCP

Q6 D2 Multi-Agent Research System CORRECT

Testing reveals that your synthesis subagent frequently needs to verify specific claims while combining findings — checking publication dates, statistics, and entity names. Currently, verification requires returning control to the coordinator, which invokes the web search subagent, then re-invokes synthesis with results. This adds 2–3 coordinator round-trips per task and increases end-to-end latency by 40%. Evaluation shows 85% of verifications are simple fact-checks while 15% require deeper investigation. What most effectively reduces overhead while maintaining research reliability?

A Give the synthesis subagent access to all web search tools so it can handle any verification need directly without coordinator round-trips.

B Give the synthesis subagent a scoped `verify_fact` tool for simple lookups, while complex verifications continue routing through the coordinator to the web search subagent.

C Have the synthesis subagent accumulate all verification needs and return them as a batch to the coordinator at the end of its pass, which routes them all to the web search subagent at once.

D Have the web search subagent proactively cache extra context around each source during initial research, anticipating what the synthesis subagent might need to verify.

WHY THE OTHER CHOICES MISS

OPTION A

Giving the synthesis subagent all web search tools expands its scope beyond its role — it would need to reason about search strategies, source selection, and depth of investigation, which belongs with the web search specialist.

CORRECT RATIONALE (OPTION B)

Task Statement 2.3 documents providing scoped cross-role tools for high-frequency, low-complexity needs. A `verify_fact` tool handles the 85% of simple lookups directly — no coordinator round-trips — while the 15% requiring deeper investigation still route through the coordinator to the specialist web search subagent. This preserves reliability for complex cases while eliminating unnecessary latency for the common case.

OPTION C

Batching verifications at end-of-pass delays synthesis completion and requires holding unverified claims in working memory throughout the pass — the benefit comes from eliminating round-trips, not batching them.

OPTION D

Pre-caching requires the web search subagent to predict what synthesis will need, which is speculative and increases initial search cost rather than reducing verification overhead.

PRACTICE TAKEAWAY

Guiding principle: *Scoped cross-role tools handle high-frequency simple operations without violating role boundaries or adding coordinator round-trips.*
Design guidance: *Provide synthesis agents with constrained tools (e.g., `verify_fact` restricted to simple lookups) for common in-pass operations, routing*

complex cases through the coordinator to the appropriate specialist.

Review topic: Tool Design & MCP

Q7 D1 Multi-Agent Research System CORRECT

A researcher runs your system on the topic "economic impacts of remote work" and receives a report covering only productivity studies, commute cost savings, and office space reduction. The report omits mental health effects, labour mobility patterns, and urban economic shifts entirely. Coordinator logs show exactly three subtasks were assigned: "remote work productivity studies," "remote work commute cost savings," and "remote work office space reduction." Each subagent returned high-quality results within its assigned scope. What is the root cause?

A The synthesis subagent lacks instructions for identifying coverage gaps in the findings it receives, so it cannot flag when major topic areas are absent from the assembled evidence.

B The web search subagent's queries were too specific, and broader query strategies would have surfaced sources covering the omitted topic areas alongside the targeted ones.

C The document analysis subagent applied overly strict relevance scoring calibrated for economic cost-benefit content, filtering out mental health and labour mobility sources as off-topic.

D The coordinator's task decomposition was too narrow.

WHY THE OTHER CHOICES MISS

OPTION A

Synthesis gap detection is useful, but the logs show root cause is coordinator decomposition, not synthesis blindness.

OPTION B

The web search subagent ran exactly the queries it was assigned.

OPTION C

Document analysis filtered based on its assigned scope; the scope itself was the problem.

CORRECT RATIONALE (OPTION D)

Task Statement 1.2 documents the risk of overly narrow task decomposition by the coordinator leading to incomplete coverage of broad research topics. The logs directly show the coordinator decomposed the topic into only three narrow economic subtopics, completely

omitting major impact categories. The subagents executed correctly within their assigned scope.

PRACTICE TAKEAWAY

Guiding principle: *High-quality subagent work cannot recover topic areas the coordinator never assigned.*
Design guidance: *Have the coordinator plan broad coverage before delegation and validate the task list for omitted dimensions before spawning subagents.*

Review topic: Agentic Architecture

Q8 D5 Multi-Agent Research System CORRECT

Your multi-agent research system produces a polished 12-page executive report combining findings from three subagents. During one run the academic database subagent fails due to a service outage and returns no data. The final report is delivered to the client without any indication that one third of the research sources were unavailable — the client assumes the report reflects complete evidence coverage. What output design prevents this misrepresentation?

A Structure the output with coverage annotations.

B Terminate the synthesis task and return an error to the client if any subagent's data is unavailable.

C Queue the synthesis task via the Message Batches API for up to 24 hours so the academic database subagent has time to recover before synthesis begins on the complete dataset.

D Have the synthesis subagent fill gaps using its parametric knowledge and label those sections as "model-estimated" rather than "source-retrieved" to distinguish the two types of content.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Task Statement 5.3 documents structuring synthesis output with coverage annotations indicating which findings are well-supported versus which

OPTION B

Terminating on partial availability is overly conservative; the system can produce useful partial results with appropriate gap annotations.

topic areas have gaps due to unavailable sources. This prevents the coordinator from producing misleading complete-looking reports when data is missing.

OPTION C

The Message Batches API does not guarantee service restoration and does not support multi-turn agentic workflows.

OPTION D

Synthesising from parametric knowledge introduces hallucination risk and misrepresents provenance of data never actually retrieved.

PRACTICE TAKEAWAY

Guiding principle: *Degrade gracefully with transparency: annotate which findings are well-supported and which have gaps.*
Design guidance: *Add coverage annotations to synthesis output so missing-source gaps are visible rather than hidden.*

Review topic: Context Management

Q9 D1 Multi-Agent Research System CORRECT

Your research coordinator dispatches subagents sequentially: it sends a query to the web search subagent, waits for its results, then sends a query to the document analysis subagent, then waits again. End-to-end latency on research queries averages 4.2 seconds. Profiling shows the web search subagent takes 1.8 seconds and the document analysis subagent takes 1.9 seconds, and the two tasks research entirely independent aspects of the topic. What change most effectively reduces latency?

A Increase `max_tokens` for each subagent so they produce fuller outputs in fewer turns, reducing the total number of subagent invocations per research query.

B Emit both Task tool calls in a single coordinator response so the SDK executes the web search and document analysis subagents concurrently rather than sequentially.

C Configure a shorter timeout on the web search subagent so the document analysis subagent can begin processing sooner, even if web search has not fully completed yet.

D Move both subagent tasks into the coordinator's context window and process all research steps in a single pass, eliminating the overhead of spawning separate subagents.

WHY THE OTHER CHOICES MISS

OPTION A

Larger max_tokens increases output size, not parallelism; it does not reduce the sequential bottleneck.

CORRECT RATIONALE (OPTION B)

Task Statement 1.3 documents spawning parallel subagents by emitting multiple Task tool calls in a single coordinator response rather than across separate turns. When both subagents research independent aspects, parallel execution eliminates sequential latency.

OPTION C

Reducing the timeout causes incomplete research rather than parallelism.

OPTION D

Moving tasks into the coordinator creates a single-agent bottleneck and eliminates the parallelism benefits of multi-agent architecture.

PRACTICE TAKEAWAY

Guiding principle: *Independent subagent tasks should run concurrently when their outputs do not depend on one another.*
Design guidance: *Emit multiple Task calls in the same coordinator turn for independent searches, reviews, or generation tasks, then aggregate the results.*

Review topic: Agentic Architecture

Q10 D5 Multi-Agent Research System CORRECT

Your synthesis subagent combines findings from a web search subagent, a document analysis subagent, and an academic database subagent into a final cited report. Post-synthesis quality review finds that source URLs and paper citations are completely absent from the report despite each subagent clearly including source attribution in its individual outputs. Coordinator logs confirm the synthesis subagent received all three subagent outputs. What is the most likely cause?

A The synthesis subagent's context window filled before it processed all three subagent outputs, causing it to drop citation metadata from the truncated sections.

B The synthesis subagent's system prompt instructs it to write in a flowing narrative style, and the model interprets this as permission to omit inline citations in favor of prose readability.

C The search and analysis subagents returned source URLs as metadata fields outside the main findings content, and the synthesis subagent processed only the primary content field.

D The synthesis subagent received summarized findings in which claim-to-source mappings were lost during the summarization step.

WHY THE OTHER CHOICES MISS

OPTION A

Context window limits cause truncation of entire inputs, not selective dropping of citation metadata.

OPTION B

Narrative style instructions explain format preferences but do not cause source URLs already in the input to disappear from the output.

OPTION C

The issue described is not a metadata field access problem; it is citation loss during summarisation.

CORRECT RATIONALE (OPTION D)

Task Statement 5.6 identifies source attribution being lost during summarisation steps when findings are compressed without preserving claim-source mappings. The synthesis subagent needs structured claim-source mappings (not just narrative summaries) from upstream agents to preserve attribution through synthesis.

PRACTICE TAKEAWAY

Guiding principle: *Summaries can erase provenance unless claim-source links are preserved as structured data.*
Design guidance: *Require upstream subagents to return claim-source mappings and pass those mappings through synthesis instead of relying on narrative summaries alone.*

Review topic: Context Management

Q11 D2 Multi-Agent Research System INCORRECT

You gave your document-analysis subagent a general `fetch_url` tool so it could load documents from links. In production it now frequently fetches search-engine result pages to run ad-hoc web searches — work that should go through

the dedicated web-search agent — producing inconsistent results. What is the most effective fix?

A Add a sentence to the document-analysis subagent's prompt telling it to use `fetch_url` only for document links, never for searching.

B Replace `fetch_url` with a `load_document` tool that accepts only document URLs and rejects search-engine and query URLs.

C Keep `fetch_url` but add a filter that blocks calls to a hand-maintained list of known search-engine domains.

D Route every `fetch_url` call through the coordinator so it can decide whether the URL is a document or a search.

WHY THE OTHER CHOICES MISS

OPTION A

A prompt instruction is probabilistic — the subagent can still choose to fetch a search page when it seems useful.

CORRECT RATIONALE (OPTION B)

Replacing the general tool with a constrained `load_document` tool that only accepts document URLs follows least privilege (TS 2.3): it makes the unwanted search behavior impossible at the interface, rather than discouraging it. A scoped tool is more reliable than prompt rules or denylists.

OPTION C

A domain denylist is brittle and incomplete; new search endpoints slip through, and it still exposes a general fetch capability.

OPTION D

Routing every fetch through the coordinator adds a round-trip to all document loads to police a capability that should simply be scoped away.

PRACTICE TAKEAWAY

Guiding principle: *Constrain capability at the tool interface; least privilege beats prompt rules and denylists.*
Design guidance: *Replace general tools with purpose-specific ones (e.g., load_document for fetch_url) that make misuse impossible.*

Review topic: Tool Design & MCP

Review D2 in Cheat Sheet

Q12 D2 Multi-Agent Research System CORRECT

Your research coordinator wastes many tool calls discovering what data is even available — repeatedly listing the document corpus, the saved-query catalog, and the source registry before it can plan. These catalogs are large but change slowly. How can you give the agent visibility into available data without these exploratory tool calls?

A Expose the corpus, query catalog, and source registry as MCP resources so the agent can see the available data up front instead of discovering it through tool calls.

B Increase the agent's `max_tokens` so it can hold the results of all the exploratory listing calls in one response.

C Add a system-prompt paragraph telling the agent not to list the catalogs more than once per session.

D Cache the listing tool results in memory so repeated listing calls return faster on later turns within the session.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

MCP resources are the mechanism for exposing slow-changing content catalogs (document corpora, query catalogs, schemas) to the agent so it sees available data without exploratory tool calls (TS 2.4). This removes the repeated discovery round-trips at their source.

OPTION B

More output budget does not stop the agent from making the exploratory calls in the first place.

OPTION C

A prompt rule is probabilistic and still requires at least one discovery pass per catalog per session.

OPTION D

Faster cached calls are still calls; resources let the agent see the catalog without calling a tool at all.

PRACTICE TAKEAWAY

Guiding principle: *Expose slow-changing content catalogs as MCP resources to cut exploratory tool calls.*
Design guidance: *Publish corpora, catalogs, and schemas as MCP resources so the*

agent sees available data up front.

Review topic: Tool Design & MCP

Q13 D1 Multi-Agent Research System INCORRECT

In your research system the web search and document analysis subagents both finish successfully, but the synthesis subagent's report only ever reflects the document-analysis findings — the web results are missing. Coordinator logs show synthesis was spawned with only the document-analysis output in its prompt. What is the root cause?

A The synthesis subagent automatically inherits the coordinator's context, so the web results should already be present and something deleted them.

B The coordinator did not include the web search subagent's output in the synthesis subagent's prompt, and subagents do not share context automatically.

C The web search subagent returned its results in a metadata field that the synthesis subagent's system prompt ignores.

D The synthesis subagent ran out of context and dropped the web results because they appeared earlier in its prompt than the document findings.

WHY THE OTHER CHOICES MISS

OPTION A

There is no automatic context inheritance for subagents; that assumption is exactly the misconception causing the bug.

CORRECT RATIONALE (OPTION B)

Subagents are isolated and do not inherit the coordinator's or each other's context — each receives only what the coordinator explicitly puts in its prompt (TS 1.3). The logs show synthesis was given only the document-analysis output, so the web findings were never passed in.

OPTION C

A metadata-field mismatch would be a parsing issue, but the logs show the web output was not placed in the synthesis prompt at all.

OPTION D

Context exhaustion would truncate broadly; here the web results were simply never included in the prompt.

PRACTICE TAKEAWAY

Guiding principle: *Subagents are isolated; the coordinator must forward every needed upstream output explicitly.*
Design guidance: *Aggregate all subagent outputs and pass them into the synthesis prompt with source attribution.*

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q14 D1 Multi-Agent Research System CORRECT

Your research coordinator runs a single pass: it spawns search and analysis subagents, then synthesizes whatever they return. On broad topics the synthesis frequently has gaps — whole subtopics are thin or missing — but the coordinator reports done anyway. What coordination pattern most directly improves coverage?

A Spawn more search subagents up front on every run so there is more raw material for synthesis regardless of the topic.

B Have the coordinator evaluate the synthesis for coverage gaps and re-delegate targeted search and analysis tasks until coverage is sufficient.

C Increase each subagent's `max_tokens` so they return more content per call and the synthesis has more to work with.

D Let the synthesis subagent flag gaps in its final output so the human reader knows which subtopics are thin.

WHY THE OTHER CHOICES MISS

OPTION A

Blindly spawning more subagents wastes effort on every run and still has no mechanism to detect and fill the specific gaps.

CORRECT RATIONALE (OPTION B)

An iterative refinement loop — the coordinator evaluates the synthesis for gaps and re-delegates targeted subtasks until coverage is sufficient — directly addresses incomplete coverage (TS 1.2). A single fixed pass cannot recover subtopics that the first round under-covered.

OPTION C

More tokens per subagent yields longer outputs, not broader topic coverage decided by the coordinator.

OPTION D

Flagging gaps for the reader documents the problem but does not fix coverage; the coordinator should close the gaps before finishing.

PRACTICE TAKEAWAY

Guiding principle: *Coordinators should evaluate synthesis for gaps and re-delegate until coverage is sufficient.*
Design guidance: *Run an iterative refinement loop: detect coverage gaps, re-delegate targeted subtasks, repeat until complete.*

Review topic: Agentic Architecture

Q15 D3 Multi-Agent Research System INCORRECT

Your research system has three specialized subagents with distinct roles: a web search subagent (authorized for `web_search` and `fetch_url`), a document analysis subagent (authorized for `read_document` and `extract_entities`), and a synthesis subagent (authorized for `write_report`). How should tool restrictions be configured to prevent each subagent from calling out-of-role tools?

A Add a shared CLAUDE.md rule to each subagent listing the tools it is permitted to use, with its role name included for context so the model knows which rules apply to it.

B Configure `allowedTools` separately in each subagent's instantiation.

C Add a coordinator-level PreToolUse hook that intercepts all subagent tool calls and blocks any that fall outside the permitted set for that subagent's assigned role.

D Configure separate MCP server namespaces per subagent type so the MCP server only exposes the appropriate tools to each namespace connection at the server level.

WHY THE OTHER CHOICES MISS

OPTION A

CLAUDE.md instructions are soft constraints — subagents could still attempt out-of-role tool calls.

CORRECT RATIONALE (OPTION B)

Each subagent is an independent instantiation with its own configuration. Setting `allowedTools` per subagent ensures each can only access its designated tools — a web search

subagent cannot call `write_report` even if instructed to.

OPTION C

The coordinator cannot intercept tool calls made within independent subagent sessions.

OPTION D

Separate MCP namespaces require server-side changes; `allowedTools` per instantiation achieves the same at the configuration layer.

PRACTICE TAKEAWAY

Guiding principle: *Agents choose tools more reliably when each role sees only the tools it needs.*
Design guidance: *Partition broad tool sets across specialized subagents instead of forcing one coordinator to choose among many unrelated tools.*

Review topic: Claude Code Config

Review D3 in Cheat Sheet

Q16 D2 Customer Support Resolution Agent CORRECT

Your support agent should call a tool on every turn — never reply with free-form text first — because each customer turn should trigger a lookup or action, yet the model occasionally returns a chatty paragraph and no tool call, stalling the flow. Separately, on the refund path `verify_identity` must run before any other tool. Which configuration enforces both behaviours?

A Set `tool_choice: "any"` so the model must call some tool each turn, and on the refund path force `tool_choice: {"type":"tool","name":"verify_identity"}` to run it first.

B Keep `tool_choice: "auto"` and add a system-prompt rule telling the model to always call a tool and to verify identity before any refund.

C Lower the sampling temperature to zero so the model deterministically prefers calling tools over returning conversational text.

D Add a PostToolUse hook that rejects any turn where the model returned plain text instead of a tool call and forces a retry.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

`tool_choice: "any"` guarantees the model calls a tool rather than returning conversational text, and a forced `{"type":"tool","name":"verify_identity"}` guarantees that specific tool runs first (TS 2.3). Both are deterministic API-level controls, unlike prompt or sampling tweaks.

OPTION B

`auto` lets the model choose text, and a system-prompt rule is probabilistic — exactly the behaviour that is already failing.

OPTION C

Temperature affects sampling randomness, not whether a tool is called or which tool runs first.

OPTION D

A hook reacts after the model already returned text, adding retry latency; tool_choice prevents the bad turn at the API level.

PRACTICE TAKEAWAY

Guiding principle: *Scope each role's tools and force tool use when free-form text is unacceptable.*
Design guidance: *Use tool_choice "any" to guarantee a tool call, and a forced {type:tool,name} to run a specific tool first.*

Review topic: Tool Design & MCP

Q17 D4 Customer Support Resolution Agent INCORRECT

Your support agent generates handoff case summaries for internal tickets. QA reviewed 50 summaries and found 72% omitted the refund amount, 64% omitted the original charge date, and 58% omitted which resolution steps had already been attempted. You already added "write a complete case summary" to the prompt six weeks ago with no measurable improvement. What is the most effective fix?

A Provide an output template listing required fields with placeholder labels: Customer ID, Order number, Charge amount and date, Issue type, Steps already attempted, and Recommended next action.

B Provide 3 few-shot examples showing a poor summary followed by a corrected version that includes all required fields, demonstrating what completeness looks like in practice.

C Switch to structured JSON output with a required-field schema: `customer_id`, `order_number`, `charge_amount`, `charge_date`, `steps_attempted`, and `recommended_action`

D Add the instruction "Be thorough and do not omit any relevant case details" and add a post-processing check that alerts on summaries lacking a dollar amount or date string.

WHY THE OTHER CHOICES MISS

OPTION A

An output template is a softer constraint — the model can acknowledge the fields yet still omit values in prose. JSON schema enforcement is the documented pattern.

OPTION B

Few-shot examples demonstrate completeness but do not enforce it — the model may still omit fields in novel cases.

CORRECT RATIONALE (OPTION C)

A required-field JSON schema makes omission a structural failure, not a judgement call — the model cannot produce valid output without populating every field. "Complete" is subjective; a schema is not.

OPTION D

"Thorough" is the same vague instruction that already failed; character-presence post-processing misses amounts spelled out in prose.

PRACTICE TAKEAWAY

Guiding principle: *Schema constraints prevent structural omissions but must match what the source can actually support.*

Design guidance: *Use required fields for mandatory data, nullable fields for absent source data, and validation/retry only when the information can exist.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q18 D2 Customer Support Resolution Agent INCORRECT

Your compliance requirement states that `verify_identity` must run before every `process_refund` call. A senior engineer proposes a PostToolUse hook on `get_customer` that triggers identity verification immediately after customer data is retrieved. QA immediately spots a gap. What is the flaw?

A The PostToolUse hook on `get_customer` fires only when called.

B PostToolUse hooks add 200–400 ms of synchronous latency to every `get_customer` call, including read-only lookups that have no downstream financial consequence.

C The hook verifies identity on every `get_customer` call regardless of whether the session will ever reach `process_refund`, creating unnecessary friction for informational queries.

D PostToolUse hooks execute in an isolated context and cannot call external verification services.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

The requirement is that `verify_identity` runs before every `process_refund`. A PostToolUse hook on `get_customer` only fires when `get_customer` is called — if `process_refund` is reached without a prior `get_customer` call, the verification is skipped. A PreToolUse hook on `process_refund` itself is the correct pattern.

OPTION B

Hook latency is manageable and not the architectural flaw described.

OPTION C

Over-verification is a UX concern, not a compliance gap — the real risk is under-verification.

OPTION D

PostToolUse hooks can perform external service calls.

PRACTICE TAKEAWAY

Guiding principle: *Business-critical tool rules need hard enforcement at the workflow or hook layer.*
Design guidance: *Use prerequisite gates or PreToolUse hooks when a tool call must be blocked unless a policy, approval, or prior result exists.*

Review topic: Tool Design & MCP

Review D2 in Cheat Sheet

Q19 D5 Customer Support Resolution Agent INCORRECT

After 18 turns your agent has confirmed: customer ID C-4421, order #55201, a double charge of $89.99 on March 14, verified refund eligibility, and one prior failed retry attempt. The context window is 60% full and the dispute may require 15 more turns. Which approach best preserves these specific facts without losing precision?

A Use progressive summarization to compress turns 1-15 into a concise narrative paragraph, retaining only turns 16-18 at full resolution to stay within context limits.

B Extract the confirmed transactional facts into a structured case-facts block at the top of each subsequent prompt, keeping them outside the rolling conversation history.

C Use `/compact` to compress the full conversation history and rely on the summarization model to identify and retain the key facts in its compressed representation.

D Ask the customer to confirm the account number and charge amount again so they appear in recent turns where model attention is highest rather than buried in turn 3 and turn 5.

WHY THE OTHER CHOICES MISS

OPTION A

Narrative summarization loses precision on amounts, dates, and order IDs.

CORRECT RATIONALE (OPTION B)

Structured case-facts at the start of each prompt place critical data in the highest-attention zone. Progressive summarization often condenses specific numbers ("$89.99 double charge on March 14") into vague descriptions ("a billing dispute").

OPTION C

`/compact` compresses history but the summary may generalize the specific facts needed for dispute resolution.

OPTION D

Re-asking the customer repeats work already done and degrades the experience.

PRACTICE TAKEAWAY

Guiding principle: *Critical facts should be promoted into a stable structure instead of buried in long chat history.*

Design guidance: *Maintain a case-facts block containing IDs, dates, amounts, decisions, and failed attempts, and place it near the start of subsequent prompts.*

Review topic: Context Management

Review D5 in Cheat Sheet

Q20 D1 Customer Support Resolution Agent CORRECT

Your support agent resolves simple requests ("refund order #1234") in 3–4 tool calls at 92% success, but on multi-concern messages ("I was double-charged, my discount didn't apply, and I want to cancel") it averages 12+ tool calls at 54% success — investigating each concern sequentially and re-fetching the same customer record for every one. What change most improves the complex cases?

A Decompose the message into distinct concerns, then investigate them in parallel against one shared customer context before synthesizing a single resolution.

B Add few-shot examples of ideal tool-call sequences for common multi-part billing messages to the system prompt.

C Consolidate get_customer, lookup_order, and billing lookups into one investigate_issue tool to reduce the number of calls.

D Add verification gates that force the agent to fully checkpoint each concern before it is allowed to begin the next one.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Decomposing the request into distinct concerns and investigating them in parallel against one shared customer context (TS 1.2) removes both failure drivers: the redundant per-concern re-fetching and the serial round-trips. The agent then synthesizes a single resolution from the shared findings.

OPTION B

Few-shot sequences improve ordering but do not eliminate the redundant customer re-fetch or the serial investigation pattern that inflates tool calls.

OPTION C

Consolidating tools reduces call count but does not parallelize the work or stop the agent from mishandling multiple concerns.

OPTION D

Forcing a checkpoint between concerns makes the flow more sequential, worsening the latency the question is trying to reduce.

PRACTICE TAKEAWAY

Guiding principle: *Decompose independent concerns and investigate them in parallel against shared context.*
Design guidance: *Split a multi-concern request into distinct items, reuse one shared context across them, then synthesize a unified resolution.*

Review topic: Agentic Architecture

Q21 D1 Customer Support Resolution Agent CORRECT

Your compliance team requires that no refund over $500 be processed without manager approval. A system prompt rule states this. Over 30 days QA reviewed 250 high-value refund cases and found 10 were processed without approval — a 4% violation rate. A junior engineer proposes adding few-shot examples showing correct escalation at $250, $500, $501, and $750. What is the correct response?

A Reject it. Implement a PreToolUse hook that intercepts `process_refund`, reads the `amount` field, and blocks execution when the value exceeds $500.

B Accept it as a short-term patch while also enabling a PostToolUse hook that logs all `process_refund` calls above $500 to a compliance audit queue for human review.

C Reject it. Remove `process_refund` from `allowedTools` entirely so all refunds, regardless of amount, must go through the human escalation path.

D Accept it. Few-shot examples directly at the $500 threshold are the highest-signal technique for boundary compliance and will eliminate the violation rate.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Prompt instructions and few-shot examples are probabilistic. A 4% failure rate is unacceptable for a financial compliance requirement. Only deterministic enforcement via a PreToolUse hook guarantees zero bypasses.

OPTION B

PostToolUse logging audits violations after they happen — it doesn't prevent them.

OPTION C

Removing the tool breaks all refund processing, including compliant ones

OPTION D

Few-shot examples may reduce the rate but cannot eliminate it.

under $500.

PRACTICE TAKEAWAY

Guiding principle: *Business-critical tool rules need hard enforcement at the workflow or hook layer.*
Design guidance: *Use prerequisite gates or PreToolUse hooks when a tool call must be blocked unless a policy, approval, or prior result exists.*

Review topic: Agentic Architecture

Q22 D2 Customer Support Resolution Agent CORRECT

Production logs show your support agent calls `get_customer` when users ask about orders ("check my order #12345") instead of `lookup_order`. Both tools have one-line descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What is the most effective first step to improve tool selection?

A Merge both tools into a single `lookup_entity` tool that accepts any identifier and decides internally which backend to query.

B Add a routing layer that inspects each message for keywords and identifier patterns and pre-selects the tool before the model runs.

C Expand each tool's description with the input formats it handles, example queries, and explicit boundaries for when to use it versus the other.

D Add 6–8 few-shot examples to the system prompt showing order-style queries routing to `lookup_order`.

WHY THE OTHER CHOICES MISS

OPTION A

Consolidating into one tool is a larger architectural change than a "first step" warrants and hides the routing decision inside opaque tool logic.

OPTION B

A keyword routing layer bypasses the model's native selection and is brittle to phrasing — it treats the symptom rather than the inadequate descriptions.

CORRECT RATIONALE (OPTION C)

Tool descriptions are the primary signal the model uses to select tools (TS 2.1).

OPTION D

Few-shot examples add token overhead without fixing the root cause:

Minimal, near-identical descriptions leave the model unable to distinguish the two; expanding each with input formats, example queries, and use/do-not-use boundaries is the low-effort, high-leverage first step.

descriptions that fail to differentiate the tools.

PRACTICE TAKEAWAY

Guiding principle: *Tool descriptions are the primary selection signal; differentiate purpose, inputs, and boundaries.*
Design guidance: *Expand overlapping tool descriptions with input formats, example queries, and explicit use/do-not-use boundaries.*

Review topic: Tool Design & MCP

Q23 D1 Customer Support Resolution Agent INCORRECT

Your support agent runs an agentic loop: after each Claude response your code either executes the requested tools and calls Claude again, or stops and replies to the customer. To cap cost, an engineer hard-codes the loop to stop after exactly 5 iterations. Production shows 23% of refund cases stop mid-investigation — the agent still had tools to call but the loop cut it off. What is the correct loop-termination design?

A Raise the iteration cap from 5 to 12 so complex refund cases have enough turns to finish before the loop stops.

B Continue the loop while `stop_reason` is `"tool_use"`, executing the requested tools and appending their results, and stop only when `stop_reason` is `"end_turn"`.

C Parse the assistant's reply for closing phrases like "is there anything else" and end the loop when one of them appears.

D Stop the loop as soon as the agent emits any natural-language text alongside its tool calls, treating that text as a completion signal.

WHY THE OTHER CHOICES MISS

OPTION A

A higher cap just moves the arbitrary cutoff; any fixed count still truncates the cases that need one more turn.

CORRECT RATIONALE (OPTION B)

The structured `stop_reason` field is the reliable termination signal (TS 1.1): continue on `"tool_use"`, stop on

`"end_turn"`. A fixed iteration cap as the sole stopping mechanism is an anti-pattern that truncates cases needing more turns — exactly the 23% failure.

OPTION C

Parsing natural-language phrases is fragile and is an explicit anti-pattern — phrasing varies and is not a reliable completion signal.

OPTION D

Assistant text often co-occurs with `tool_use`; treating any text as completion ends the loop prematurely.

PRACTICE TAKEAWAY

Guiding principle: *Agent loops terminate on structured stop signals, not iteration caps or natural-language cues.*
Design guidance: *Continue on stop_reason "tool_use", append tool results, and stop only on "end_turn".*

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q24 D3 Customer Support Resolution Agent CORRECT

Your customer support agent has access to 8 MCP tools: `lookup_customer`, `get_order_history`, `process_refund`, `update_shipping`, `escalate_ticket`, `send_email`, `modify_account`, and `delete_account`. Tier-1 agents are authorized only to look up data and escalate — they must never process refunds or modify accounts. Which configuration correctly enforces this boundary?

A Add a CLAUDE.md instruction listing the tools Tier-1 agents must not call: "Do not use process_refund, modify_account, or delete_account"

B Set `allowedTools: ["lookup_customer","get_order_history","escalate_ticket","send_email"]`

C Add a PreToolUse hook that intercepts calls to restricted tools and returns an access-denied error, preventing execution and logging the unauthorized attempt.

D Add a system prompt instruction: "As a Tier-1 agent, do not call process_refund, modify_account, or delete_account under any circumstances."

WHY THE OTHER CHOICES MISS

OPTION A

A CLAUDE.md instruction is prompt-based and therefore a soft constraint — a persuasive user message or an edge-case in the agent's reasoning could still cause it to call the restricted tools. It is not enforced at the API level.

CORRECT RATIONALE (OPTION B)

`allowedTools` is an allowlist — only listed tools can be called, and new tools added to the server later are denied by default. It enforces access at the configuration layer regardless of prompt instructions.

OPTION C

A PreToolUse hook intercepts calls after the agent decides to make them; `allowedTools` prevents that decision entirely.

OPTION D

System prompt instructions are equally soft — they can be overridden by context or user messages and do not provide the deterministic enforcement that `allowedTools` does.

PRACTICE TAKEAWAY

Guiding principle: *Tool authorization belongs in configuration, not in persuasive instructions.*
Design guidance: *Use `allowedTools` as an allowlist for each role so future tools are denied by default unless explicitly granted.*

Review topic: Claude Code Config

Q25 D1 Customer Support Resolution Agent CORRECT

Production logs reveal that the customer support agent misinterprets data returned by your MCP tools: `get_customer` returns Unix timestamps, `lookup_order` returns ISO 8601 dates, and `process_refund` returns numeric status codes (1=pending, 2=shipped). Several of these tools are third-party MCP servers you cannot modify. Agents repeatedly misread the status codes, triggering incorrect refund actions. What is the most maintainable approach to ensure the agent always receives normalized data formats?

A Add detailed format documentation to the system prompt explaining each tool's data conventions and how to interpret them.

B Use a PostToolUse hook to intercept all tool results and apply formatting transformations before the agent processes them.

C Create a `normalize_data` tool that the agent calls after each data retrieval to transform values before acting on them.

D Modify the tools you control to return human-readable formats; create wrapper MCP tools for third-party tools you cannot change.

WHY THE OTHER CHOICES MISS

OPTION A

System prompt documentation describes the formats but still requires the model to interpret and convert them correctly on every call — a probabilistic approach that fails on edge cases and novel inputs.

CORRECT RATIONALE (OPTION B)

Task Statement 1.5 identifies PostToolUse hooks as the correct pattern for intercepting and normalizing heterogeneous tool outputs before the agent processes them. A centralized hook applies transformations uniformly across all tools — including third-party MCP servers you cannot modify — without relying on the model to correctly interpret format documentation on every call.

OPTION C

A normalize_data tool relies on the agent to consistently call it after every data retrieval — the agent may skip it or call it inconsistently, making normalization probabilistic rather than guaranteed.

OPTION D

Modifying tools you control solves half the problem, but third-party MCP servers cannot be changed, and wrapper tools add architectural complexity when a PostToolUse hook already intercepts at the right layer.

PRACTICE TAKEAWAY

Guiding principle: *Data normalization across heterogeneous MCP tools belongs in a hook layer, not in prompt instructions or model-driven tool calls.*

Design guidance: *Implement PostToolUse hooks that normalize timestamps, status codes, and other format variations centrally, before the agent processes any tool result.*

Review topic: Agentic Architecture

Q26 D1 Customer Support Resolution Agent INCORRECT

Production logs show the customer support agent calls `process_refund` in 18% of sessions without a prior `get_order_history` call. Post-incident review confirms

these refunds were issued for incorrect amounts — the agent processed refunds without first verifying which order was disputed. A senior engineer proposes adding a system prompt rule: "Always call get_order_history before calling process_refund." After two weeks this reduces the violation rate to 11%, not zero. What architectural change reliably enforces the correct call sequence?

A Add 5 few-shot examples in the system prompt demonstrating the correct sequence: get_order_history call followed by process_refund with the retrieved order data as context.

B Add a system prompt instruction with emphatic language: "CRITICAL — You MUST call get_order_history before process_refund under all circumstances."

C Add a programmatic prerequisite gate: track whether get_order_history has returned for the current session, and block any process_refund call until that condition is satisfied.

D Add a PostToolUse hook on get_order_history that logs order data to the session state, so there is a record of the order history call for each session.

WHY THE OTHER CHOICES MISS

OPTION A

Few-shot examples reduce violation rate but cannot eliminate it — the existing system prompt instruction already reduced it to 11%, and examples are equally probabilistic.

OPTION B

Emphatic prompt language is also probabilistic — if the system prompt instruction reduced violations to 11%, stronger wording will not reach 0%.

CORRECT RATIONALE (OPTION C)

A structural prerequisite gate enforced in the agentic loop is deterministic — it blocks process_refund regardless of what the model decides, eliminating the residual violation rate that prompt instructions leave. This is the TS 1.4 pattern: multi-step tool workflows with structural gates that enforce sequence at the architecture layer.

OPTION D

A PostToolUse hook on get_order_history logs data after the call but does not gate or block process_refund — it provides no enforcement.

PRACTICE TAKEAWAY

Guiding principle: *Compliance-critical tool sequences need structural enforcement, not stronger wording.*

Design guidance: *Track prerequisites in the agent loop or hook layer and block*

downstream tools until required upstream tool results exist.

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q27 D2 Customer Support Resolution Agent CORRECT

The `lookup_customer` tool returns `"multiple_matches"` as a plain error string whenever a query matches more than one customer record. Analysis of 3 weeks of logs shows the agent retries the same lookup with identical parameters in 89% of these cases, creating an effective infinite loop that resolves only when a timeout fires after 4–6 additional API calls. What is the correct fix to the tool's error response?

A Expand lookup_customer's description to say: "Returns an error string 'multiple_matches' if the query matches more than one customer.

B Return a structured error: `{"isError": true, "errorCategory": "ambiguous_input", "isRetryable": false, "candidates": [{"id": "C-001", "email": "e.m@example.com"}, {"id": "C-002", "email": "e.martinez@example.com"}]}`

C Add a PreToolUse hook that normalizes all customer query strings to lowercase and strips punctuation before passing them to lookup_customer, preventing some multi-match scenarios upstream.

D Add a system prompt instruction: "When lookup_customer returns multiple_matches, try searching with a more specific query such as the customer's full name or account number."

WHY THE OTHER CHOICES MISS

OPTION A

Expanding the tool description with the error string explains what happened but still leaves the agent without recovery data — it has no candidates to present and will still guess at refinements.

CORRECT RATIONALE (OPTION B)

The structured error response with `isRetryable: false` tells the agent that retrying with the same parameters will not succeed, and the `candidates` array provides the recovery data needed to disambiguate — the agent can present the top matches to the customer and ask them to confirm. This is the TS 2.2 pattern: structured error responses with `errorCategory`, `isRetryable`, and recovery metadata.

OPTION C

A PreToolUse normalization hook reduces some multi-match scenarios but does not eliminate them — the tool still needs a correct error response for the cases that remain.

OPTION D

A system prompt instruction to refine the query is probabilistic and does not give the agent the candidate list it needs to actually resolve the ambiguity.

PRACTICE TAKEAWAY

Guiding principle: *Agents retry what they cannot classify; ambiguous-result errors should carry the recovery data needed to disambiguate.*
Design guidance: *Return isError, errorCategory, isRetryable, and a candidate list so the agent can ask the customer to confirm the right match instead of guessing or re-querying.*

Review topic: Tool Design & MCP

Q28 D5 Customer Support Resolution Agent CORRECT

After calling get_customer and lookup_order, your agent has retrieved all available system data but must decide whether to resolve autonomously or call escalate_to_human. Which situation is the most appropriate trigger for escalation?

A A customer asks to cancel an order that shipped yesterday and is out for delivery tomorrow; escalate because they may change their mind once it arrives.

B A customer requests a price match against a competitor; your policy allows adjustments for price drops on your own site within 14 days but is silent on competitor pricing, so escalate for policy interpretation.

C A customer's message raises both a billing question and a return; escalate so a human can handle the two topics together in one interaction.

D Tracking shows the order was delivered and signed for, but the customer says it never arrived; escalate because contradicting the customer could harm the relationship.

WHY THE OTHER CHOICES MISS

OPTION A

A cancel-versus-shipping timing question is handleable from the order data; "might

CORRECT RATIONALE (OPTION B)

A genuine policy gap — the policy covers own-site price drops but is silent on

change their mind" is not an escalation trigger.

competitor matching — requires human judgment, and the agent must not fabricate a policy (TS 5.2). The other cases are within the agent's data and authority to resolve.

OPTION C

Multiple topics in one message is not a reason to escalate; the agent can address both concerns itself.

OPTION D

Contradictory delivery evidence is handleable with the tracking data; worry about the relationship is not a policy gap.

PRACTICE TAKEAWAY

Guiding principle: *Escalate on genuine policy gaps, explicit human requests, and stalls — not on routine complexity.*
Design guidance: *Give explicit escalation criteria with examples, and escalate when policy is silent on the customer's specific request rather than fabricating a rule.*

Review topic: Context Management

Q29 D1 Customer Support Resolution Agent CORRECT

A customer support agent handles a complex complaint that requires three sequential steps: verify customer identity, retrieve full order history, and check refund eligibility against policy. Currently these three steps are handled in a single agentic turn. QA analysis of 80 multi-condition refund cases shows 71% accuracy — the agent frequently applies the wrong eligibility rule or skips identity verification entirely when handling all three tasks at once. What architectural approach best improves accuracy on these complex cases?

A Increase max_tokens so the agent has more reasoning budget to handle all three steps simultaneously without truncating its policy-checking logic.

B Add all three steps' logic into one expanded system prompt with detailed instructions for each phase, giving the agent a comprehensive reference in a single pass.

C Add 5 few-shot examples that demonstrate all three steps being performed correctly in sequence across different complaint types to anchor the correct multi-step behavior.

D Decompose into a sequential prompt chain: a focused identity-verification subagent, then a focused order-retrieval subagent, then a focused eligibility-check subagent.

WHY THE OTHER CHOICES MISS

OPTION A

More tokens in a single turn gives more space but does not change that the model must simultaneously track identity verification, order history, and policy rules — attention dilution persists.

OPTION B

A comprehensive single-prompt approach with more detail is still a single multi-task prompt — it does not decompose the cognitive load into focused single-task steps.

OPTION C

Few-shot examples may improve performance but are still applied within the same single-turn multi-task context; decomposition is the structural fix TS 1.6 recommends.

CORRECT RATIONALE (OPTION D)

Task Statement 1.6 documents that sequential prompt chains with single-purpose steps outperform multi-task prompts — each focused subagent applies full attention to one task rather than dividing attention across three. The 71% accuracy reflects the cost of multi-task prompting for a workflow where steps are distinct and sequential.

PRACTICE TAKEAWAY

Guiding principle: *Single-purpose chain steps with a validation boundary outperform one multi-task prompt.*
Design guidance: *Decompose a multi-condition task into focused sequential steps so each step applies full attention to one job rather than dividing it.*

Review topic: Agentic Architecture

Q30 D2 Customer Support Resolution Agent CORRECT

`process_refund` returns `{"success": false, "message": "Cannot process"}` for every failure mode. When a customer attempts a refund on an order from 14 months ago — 11 months past the 30-day return window — the agent retries the call 3 times before giving up, adding 6–9 seconds of wasted latency and consuming unnecessary API budget. What specific change to the error response format prevents this retry behavior?

A Return `{"isError": true, "content": [{"type":"text","text":"Cannot process"}], "retryAfter": null}`

B Return `{"isError": true, "content": [{"type":"text","text":"Outside 30-day return window"}], "errorCategory": "business", "isRetryable": false}` so the agent recognises this as a non-retryable policy violation.

C Return `{"success": false, "message": "Policy violation: item outside return window. No further action is possible."}` so the agent can parse the natural language message and stop retrying.

D Add a numeric HTTP-style error code: `{"success": false, "code": 403, "message": "Cannot process"}`

WHY THE OTHER CHOICES MISS

OPTION A

`retryAfter: null` is not a recognized MCP field and doesn't communicate non-retryability clearly.

CORRECT RATIONALE (OPTION B)

`errorCategory: "business"` plus `isRetryable: false` is the documented MCP error structure. Business errors represent policy violations that will never succeed on retry.

OPTION C

Natural language parsing is fragile — phrasing varies and the agent may misinterpret it.

OPTION D

HTTP codes are not part of the MCP error protocol and the agent has no schema for interpreting them.

PRACTICE TAKEAWAY

Guiding principle: *Agents retry what they cannot classify. Tool errors should state both the failure category and retryability.*

Design guidance: *Return structured MCP errors with fields such as `errorCategory` and `isRetryable` so the agent can stop, retry, or recover appropriately.*

Review topic: Tool Design & MCP

Q31 D4 Claude Code for CI/CD INCORRECT

Your overnight "deep review" job iterates: Claude analyzes a changed file, then calls a tool to fetch related files (imports, base classes, tests), receives the results, and continues — several tool rounds per file. To cut cost you want to move it to the Message Batches API. What is the primary technical constraint?

A Batch processing latency of up to 24 hours is too slow even for an overnight job, so the review could never finish in time.

B The batch API's asynchronous model cannot execute a tool mid-request and feed results back for Claude to continue, which breaks the iterative tool-calling loop.

C The batch API cannot accept tool definitions in the request parameters at all, so no tools can be declared.

D The batch API lacks request identifiers, so you cannot match each response back to the file it analyzed.

WHY THE OTHER CHOICES MISS

OPTION A

A 24-hour window is acceptable for an overnight job; latency is not the blocking constraint here — the tool-calling loop is.

CORRECT RATIONALE (OPTION B)

The Batches API is fire-and-forget: there is no mechanism to intercept a tool call mid-request, run the tool, and return results for Claude to continue (TS 4.5). That fundamentally breaks multi-round tool-calling. Batch does support tool definitions and provides `custom_id` for correlating responses.

OPTION C

Batch requests can include tool definitions; the limitation is executing tools mid-request, not declaring them.

OPTION D

`custom_id` exists precisely to correlate batch responses to requests, so identification is not the problem.

PRACTICE TAKEAWAY

Guiding principle: *Match the API to the workload; batch cannot do multi-turn tool calls within one request.*
Design guidance: *Keep iterative tool-calling workflows synchronous; use batch for single-shot, latency-tolerant jobs and correlate results with custom_id.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q32 D3 Claude Code for CI/CD CORRECT

Your CI pipeline runs `claude "Review this PR" --output-format json` but the job blocks indefinitely in automated runs while returning results correctly when a developer runs the same command locally on their workstation. What is the missing flag?

A `-p` / `--print` — non-interactive mode: processes the prompt, writes output to stdout, and exits without waiting for further input.

B `--batch` — switches Claude Code to batch-processing mode designed for CI and automated pipeline contexts.

C `--headless` — instructs Claude Code to run without any interactive terminal UI elements that block in a CI environment.

D `--no-input` — disables stdin so the process does not wait for keyboard input when running in a non-TTY context.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

`-p` (`--print`) is the documented flag for non-interactive CI use. It processes the prompt and exits without waiting for input.

OPTION B

`--headless` is not a documented Claude Code flag.

OPTION C

`--no-input` is not a documented Claude Code flag.

OPTION D

`--batch` is not a documented Claude Code flag. Knowing which flags actually exist vs plausible-sounding ones is directly tested.

PRACTICE TAKEAWAY

Guiding principle: *Run Claude Code non-interactively with -p; distinguish real CLI flags from plausible fakes.*
Design guidance: *Use -p/--print to process a prompt and exit, and pair it with --output-format json and --json-schema for structured CI output.*

Review topic: Claude Code Config

Q33 D5 Claude Code for CI/CD CORRECT

A CI pipeline agent reviews 18 changed files in a single session. Developers report that findings for files 7–12 are consistently less detailed than findings for files 1–6 and files 13–18 — the middle portion of the diff receives superficial analysis with fewer flagged issues and shorter explanations. The full 18-file diff fits within the context window, no tool errors are logged, and the agent does not hit its max_turns limit. What is causing this pattern?

A The agent is reaching its max_turns limit and summarizing middle sections to complete the review within the turn budget, producing shorter output for those files.

B Files 7–12 contain more complex logic than the surrounding files, requiring more reasoning tokens per finding.

C The lost-in-the-middle effect.

D The `lint_code` tool output for files 7–12 is truncated by the MCP server's response size limit, leaving the agent with insufficient raw data to produce detailed findings for those files.

WHY THE OTHER CHOICES MISS

OPTION A

The question states the agent does not hit its max_turns limit, ruling out turn-budget summarization as the cause.

OPTION B

File complexity does not explain a position-correlated pattern — the same files 7–12 consistently receive shallower coverage across different PRs, regardless of their actual complexity.

CORRECT RATIONALE (OPTION C)

Task Statement 5.1 documents the lost-in-the-middle effect: models process content near the start and end of long contexts most reliably and give reduced attention to material in the middle. With 18 files fed sequentially, files 7–12 land in the low-attention middle region of the accumulated diff, producing consistently shallower analysis. The mitigation is splitting reviews into smaller focused batches so no file lands in the middle of a long context.

OPTION D

MCP server truncation would produce tool errors or identical short outputs for those files, not consistently shallower analysis with fewer flagged issues.

PRACTICE TAKEAWAY

Guiding principle: *Long inputs hide middle content even when nothing is truncated.*
Design guidance: *Front-load key context, add section headers, or split into smaller focused passes so no file lands in the low-attention middle.*

Review topic: Context Management

Q34 D3 Claude Code for CI/CD INCORRECT

Your CI repo's root CLAUDE.md has grown to 700 lines covering review standards, test conventions, deployment notes, and language-specific style for three stacks, and it is hard to maintain. You want to keep one authoritative set of instructions loaded for reviews but split it into separate, individually maintained files by topic. What is the idiomatic Claude Code approach?

A Break the content into topic files and reference them from the root CLAUDE.md using `@import` directives so they load together as one instruction set.

B Move every topic into a separate `.claude/skills/` SKILL.md and have the review job invoke each skill it needs at run time.

C Split the content into subdirectory CLAUDE.md files and rely on the review job running from the matching subdirectory.

D Paste the topic files into the review prompt at invocation time so the model receives all of them on every run.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

`@import` lets the root CLAUDE.md reference separately maintained topic files while still loading them as one always-on instruction set for every session (TS 3.1). It keeps a single authoritative source without a 700-line monolith.

OPTION B

Skills are on-demand and would not be reliably loaded as always-on review standards; this scatters universal standards into invocation-time steps.

OPTION C

Subdirectory CLAUDE.md files load by working directory, so one review run would not pick up topics outside its directory.

OPTION D

Pasting files into each prompt is manual and error-prone — exactly the maintenance problem @import solves.

PRACTICE TAKEAWAY

Guiding principle: *Keep CLAUDE.md modular: reference separately maintained topic files with @import.*
Design guidance: *Use @import in the root CLAUDE.md to compose topic files into one always-loaded instruction set.*

Review topic: Claude Code Config

Review D3 in Cheat Sheet

Q35 D1 Claude Code for CI/CD INCORRECT

Your CI pipeline's review agent posts inline PR comments, then immediately calls `approve_pr` without verifying that all blocking findings have been resolved. A senior engineer proposes limiting the agent to a maximum of 3 tool calls per run to force it to stop before reaching `approve_pr` prematurely. What is the fundamental problem with this solution?

A The 3-call cap is too restrictive.

B An iteration cap as the sole termination mechanism is an anti-pattern.

C The pipeline should pass `tool_choice: {"type":"any"}` at the end of each turn so the agent self-reports completion rather than having an external cap imposed on it.

D The root issue is that `approve_pr` lacks a PreToolUse hook enforcing mandatory human sign-off.

WHY THE OTHER CHOICES MISS

OPTION A

The cap value is secondary — using any fixed iteration count as the sole termination signal is the fundamental flaw.

CORRECT RATIONALE (OPTION B)

Arbitrary iteration caps as the primary termination mechanism are an explicit anti-pattern per Task Statement 1.1. The correct pattern is to check `stop_reason` and terminate the loop when the model returns `end_turn` after genuinely completing the task.

OPTION C

OPTION D

`tool_choice: "any"` forces tool use but doesn't address when the agent has actually finished its task.

A PreToolUse hook on `approve_pr` is a valid safety measure but doesn't fix the loop termination anti-pattern.

PRACTICE TAKEAWAY

Guiding principle: *Agent loops should be controlled by structured completion signals, not arbitrary caps or natural-language guesses.*
Design guidance: *Continue while the model requests tool use, append tool results each turn, and stop only when the response reaches the documented completion state.*

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q36 D3 Claude Code for CI/CD CORRECT

You are about to overhaul your CI pipeline's Claude integration: it touches the review prompt, the test-generation step, the GitHub posting layer, and the batch-vs-synchronous split, and there are several viable designs with different trade-offs. A teammate suggests just starting to edit files and adjusting as problems surface. What is the better way to begin in Claude Code?

A Start in direct execution and only switch to plan mode if you hit unexpected complexity partway through the changes.

B Enter plan mode to explore the pipeline, map the dependencies between the four areas, and agree on an approach before any files are edited.

C Start in direct execution with one comprehensive prompt that specifies exactly how all four areas should change.

D Make the changes incrementally in direct execution and let the failing CI runs reveal the right design.

WHY THE OTHER CHOICES MISS

OPTION A

Deferring plan mode ignores that the complexity is already known up front, not something that might emerge later.

CORRECT RATIONALE (OPTION B)

Plan mode is built for ambiguous, multi-area, architectural work with several valid designs (TS 3.4). Exploring and aligning on an approach before editing prevents

costly rework across the four interdependent areas. Direct execution is for clear, well-scoped changes.

OPTION C

A comprehensive upfront prompt assumes you already know the right structure across four interdependent areas without exploring them.

OPTION D

Letting failing runs reveal the design is costly trial-and-error for a change with multiple valid architectures.

PRACTICE TAKEAWAY

Guiding principle: *Use plan mode for ambiguous, multi-file, architectural work; direct execution for clear, scoped changes.*
Design guidance: *Enter plan mode to explore and align on an approach before editing when several valid designs exist.*

Review topic: Claude Code Config

Q37 D3 Claude Code for CI/CD CORRECT

Your team's CLAUDE.md has grown to 700 lines covering React component conventions, Python service patterns, Terraform module standards, and testing guidelines. Developers report that Claude applies Terraform linting rules when editing React components and suggests Python idioms in `.tf` files. What is the most maintainable fix?

A Split CLAUDE.md into separate technology files and reference them from the root CLAUDE.md using `@import` directives so all conventions load at session start.

B Move each technology's conventions into `.claude/rules/` files with YAML `paths:` glob frontmatter.

C Create a separate slash command skill for each technology that developers invoke manually when switching between React, Python, or Terraform contexts.

D Add clear section headers and a table of contents to CLAUDE.md so Claude can identify which section applies to the file currently being edited.

WHY THE OTHER CHOICES MISS

OPTION A

CORRECT RATIONALE (OPTION B)

Section headers rely on the model inferring which section applies — path scoping is deterministic.

Path-scoped rules in `.claude/rules/` with YAML glob patterns load conditionally and automatically based on the file being edited, eliminating cross-convention contamination.

OPTION C

Manual `@import` adds friction and developers forget to load the right file.

OPTION D

Manual skill invocation adds friction and doesn't scale across a large team.

PRACTICE TAKEAWAY

Guiding principle: *Large mixed-technology repos need conditional rule loading to prevent convention bleed.*
Design guidance: *Move technology-specific standards into `.claude/rules/` files with `paths` globs instead of loading every convention into every session.*

Review topic: Claude Code Config

Q38 D4 Claude Code for CI/CD CORRECT

A PR modifies 14 files across one module. Your review prompt analyzes all 14 files together in a single pass and produces uneven results: deep feedback on some files, superficial notes on others, and contradictory calls — flagging a pattern in one file while approving identical code in another. How should you restructure the review?

A Require developers to split large PRs into 3–4 file submissions before the automated review runs.

B Switch to a higher-tier model with a larger context window so all 14 files get adequate attention in one pass.

C Run the review in focused passes: analyze each file individually for local issues, then a separate integration pass examining cross-file data flow.

D Run three independent full-PR passes and only report findings that appear in at least two of the three runs.

WHY THE OTHER CHOICES MISS

OPTION A

OPTION B

Forcing developers to split PRs shifts the burden to humans and does not fix the review architecture.

A larger context window does not solve attention dilution; the model still spreads focus unevenly across 14 files.

CORRECT RATIONALE (OPTION C)

Single-pass review over many files dilutes attention, causing uneven depth and contradictory findings (TS 4.6). Per-file passes give each file consistent local analysis, and a separate integration pass handles cross-file concerns — covering both dimensions.

OPTION D

Requiring 2-of-3 consensus suppresses real issues caught only intermittently and triples cost.

PRACTICE TAKEAWAY

Guiding principle: *Split large reviews into per-file local passes plus a separate cross-file integration pass.*
Design guidance: *Analyze each file individually for local issues, then run one integration pass for cross-file data flow.*

Review topic: Prompt Engineering

Q39 D4 Claude Code for CI/CD INCORRECT

Your PR review schema requires five fields per finding: `file_path`, `line_number`, `severity` (enum: critical/high/medium/low), `description`, and `suggested_fix`. Over 500 production extractions, 28% of findings return `severity: "warning"` (not in the enum) and 28% return `suggested_fix: null`, causing downstream filtering and ticketing pipelines to fail on both cases. How should the schema be updated?

A Keep the schema unchanged and add post-processing to map `"warning"` to `"low"` severity and replace null `suggested_fix` values with an empty string before the findings reach downstream systems.

B Add `"warning"` to the severity enum and make `suggested_fix` a required field.

C Remove the severity enum entirely and accept a free-form severity string.

D Add `"warning"` to the severity enum and make `suggested_fix` nullable with a prompt instruction: "Return null for suggested_fix only when no clear remediation exists."

WHY THE OTHER CHOICES MISS

OPTION A

Post-processing hacks mask the root cause and `"warning"` → `"low"` is a semantic assumption that may misclassify findings.

OPTION B

Required `suggested_fix` forces invention for findings where no clear fix applies — hallucination over accuracy.

OPTION C

Removing the enum loses structured severity classification, making downstream filtering impossible.

CORRECT RATIONALE (OPTION D)

Adding `"warning"` fixes the enum failure. Making `suggested_fix` nullable with a prompt instruction prevents the model from fabricating fixes where none exists.

PRACTICE TAKEAWAY

Guiding principle: *Schema constraints prevent structural omissions but must match what the source can actually support.*

Design guidance: *Use required fields for mandatory data, nullable fields for absent source data, and validation/retry only when the information can exist.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q40 D4 Claude Code for CI/CD INCORRECT

Your CI system runs two types of Claude Code analysis: (1) a pre-merge code review that blocks PR merges until complete, and (2) a nightly security audit of the full codebase that developers review the next morning. Your team wants to reduce API costs by 50% using the Message Batches API. Which workflows are appropriate candidates for batch processing?

A Both workflows — the 50% cost savings apply to all Claude API calls regardless of latency requirements.

B Neither — the Message Batches API does not support the multi-turn tool-calling workflow that code review requires.

C The nightly security audit only — it runs overnight and tolerates the up-to-24-hour processing window, while the pre-merge review requires immediate results to unblock developers.

D The pre-merge review only — it processes a single PR at a time and can be chunked into individual file review requests suitable for batch submission.

WHY THE OTHER CHOICES MISS

OPTION A

Batch processing carries a 24-hour processing window with no latency guarantee — applying it to pre-merge reviews would block PR merges for up to a full day.

OPTION B

The batch API limitation is that it does not support multi-turn tool-calling within a single request, but single-turn code analysis can still use batch processing — this is not a reason to exclude both workflows.

CORRECT RATIONALE (OPTION C)

Task Statement 4.5 specifies that the Message Batches API (50% cost savings, up to 24-hour processing window, no guaranteed latency SLA) is appropriate for latency-tolerant workloads — overnight reports, weekly audits — and inappropriate for blocking workflows where developers are waiting for results. The nightly audit fits exactly; the pre-merge review does not.

OPTION D

The pre-merge review cannot use batch processing because it blocks developers who need immediate feedback — latency tolerance is the deciding factor, not whether the review is single-file or multi-file.

PRACTICE TAKEAWAY

Guiding principle: *Match API approach to latency requirements: synchronous for blocking workflows, batch for latency-tolerant overnight or weekly analysis.*
Design guidance: *Use the Message Batches API for nightly and weekly analysis where 24-hour processing is acceptable; use synchronous calls for any workflow that blocks a developer or deployment pipeline.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q41 D3 Claude Code for CI/CD INCORRECT

Your team wants a reusable `/review` skill in `.claude/skills/review/SKILL.md` that takes a target path as an argument and may only read files and post review comments — never edit code or run shell commands. Developers also keep

invoking it with no argument and getting confused output. Which frontmatter configuration addresses both needs?

A Put the skill in `.claude/commands/review.md` and document the required argument and tool limits in the project README so developers know the rules.

B Add `argument-hint` so the expected argument shows during autocomplete, and `allowed-tools` restricting the skill to read and comment operations.

C Add `context: fork` so the skill runs in isolation, and rely on the system prompt to remind the model not to edit code or run commands.

D Add a `required-args: true` flag and a `deny-tools` list naming the edit and shell tools the skill must never call.

WHY THE OTHER CHOICES MISS

OPTION A

A README documents intent but enforces nothing; the skill could still run with no argument and still call edit or shell tools.

CORRECT RATIONALE (OPTION B)

`argument-hint` surfaces the expected argument during autocomplete (fixing the missing-argument confusion) and `allowed-tools` restricts the skill to only the read/comment operations it needs (TS 3.2). Both are real SKILL.md frontmatter fields.

OPTION C

`context: fork` isolates output but does not constrain which tools run, and a prompt reminder is probabilistic rather than enforced.

OPTION D

`required-args` and `deny-tools` are not real frontmatter fields — `argument-hint` and `allowed-tools` are the documented mechanisms.

PRACTICE TAKEAWAY

Guiding principle: *Package workflows as skills and constrain them with real frontmatter, not documentation.*
Design guidance: *Use argument-hint to prompt for required arguments and allowed-tools to restrict a skill to the tools it needs.*

Review topic: Claude Code Config

Review D3 in Cheat Sheet

Q42 D2 Claude Code for CI/CD CORRECT

Your CI pipeline's `post_comment` tool has the description "Posts a review comment." It accepts `file_path`, `line_number`, and `body` as parameters. Analysis of 3 weeks of pipeline logs shows that `line_number` is omitted in 41% of calls, resulting in general PR-level comments rather than precise inline annotations. What is the most direct fix?

A Add a PreToolUse hook that checks for the presence of `line_number` before allowing the call to proceed, rejecting the call and returning an error when the field is missing.

B Make `line_number` optional in the JSON schema with a default value of null.

C Expand the description: "Posts an inline comment on a specific line. Required: file_path, line_number (integer), body. Omitting line_number posts a general PR comment."

D Add a system prompt instruction: "Always include file_path, line_number, and body when calling post_comment — never omit line_number."

WHY THE OTHER CHOICES MISS

OPTION A

A hook is a safety measure, not a root-cause fix — the agent still won't understand why the field matters.

OPTION B

Making the field optional with null default encodes the broken behavior into the schema.

CORRECT RATIONALE (OPTION C)

The root cause is that the minimal description doesn't explain what `line_number` does or what happens when it's omitted. An expanded description explaining that `line_number` anchors the comment and that omitting it changes behavior directly teaches the model when and why to include it.

OPTION D

System prompt instructions are less effective than the tool description itself when the model constructs tool call parameters.

PRACTICE TAKEAWAY

Guiding principle: *Tool parameter descriptions should explain behavioral consequences, not just list field names.*
Design guidance: *Describe what each important parameter changes in the external*

system, especially when omission silently changes scope or granularity.

Review topic: Tool Design & MCP

Q43 D3 Claude Code for CI/CD CORRECT

Your CI pipeline runs Claude Code to produce code review findings that are automatically posted as inline PR comments. The pipeline currently collects Claude's narrative output and passes it to a script that parses file paths and line numbers from prose — a fragile approach that breaks on novel phrasing. Your findings schema requires `file_path`, `line_number`, `severity`, and `description` per finding. What CLI flag combination produces reliable structured output?

A Add `--output json` to request JSON-formatted responses from Claude Code.

B Add `--format structured` and pass a field list via `--fields file_path,line_number,severity,description`.

C Add `--output-format json` combined with `--json-schema` referencing a schema file defining the required fields.

D Define the output schema inside CLAUDE.md review instructions so Claude constrains its own output format.

WHY THE OTHER CHOICES MISS

OPTION A

`--output json` is not a documented Claude Code CLI flag — it would not produce schema-constrained structured output.

OPTION B

`--format structured` and `--fields` are not documented Claude Code CLI flags.

CORRECT RATIONALE (OPTION C)

Task Statement 3.6 documents `--output-format json` combined with `--json-schema` as the CLI flag pair for enforcing structured output in CI contexts. Together they produce well-formed JSON matching the specified schema, which can be reliably parsed

OPTION D

Defining the schema in CLAUDE.md is a prompt instruction, not an output enforcement mechanism — it does not replace the CLI flags that enforce machine-parseable output.

and posted as inline PR comments without fragile prose-parsing heuristics.

PRACTICE TAKEAWAY

Guiding principle: *CI pipelines that consume Claude output need structured output enforced at the CLI level, not parsed from narrative prose.*
Design guidance: *Use* `--output-format json` *with* `--json-schema` *in CI invocations to produce machine-parseable findings that automated systems can consume without brittle text parsing.*

Review topic: Claude Code Config

Q44 D4 Claude Code for CI/CD INCORRECT

Over 6 weeks your PR review pipeline has flagged 38% of all PRs for "potential security issues." A developer survey finds that 91% of these flags are dismissed as false positives. Developers have begun ignoring the review tool entirely, including the 9% of genuine findings. You already added "only flag confirmed vulnerabilities" to the prompt four weeks ago with no measurable change. What is the most effective approach?

A Add 15 annotated examples of false positives alongside 15 genuine findings so the model learns to distinguish real vulnerabilities from code patterns that merely resemble them.

B Require a human security engineer to triage all Claude security findings before they are posted to PR authors, so only validated issues reach developers.

C Disable security review entirely and replace it with a dedicated SAST tool such as Semgrep or CodeQL, supplemented by Claude for semantic issues those tools miss.

D Identify the highest false-positive sub-categories, disable them temporarily while refining their detection criteria, then re-enable them incrementally as precision improves.

WHY THE OTHER CHOICES MISS

OPTION A

Negative examples help but are a prompt fix, not a category-level fix — the question asks for the most effective approach to a pervasive precision problem.

OPTION B

Human approval defeats the automation purpose and doesn't fix the underlying false positive problem.

OPTION C

Separate tooling adds infrastructure without fixing the prompt calibration problem, and loses Claude's semantic understanding.

CORRECT RATIONALE (OPTION D)

Task Statement 4.1 recommends this approach: disable high-noise categories to restore developer trust while iterating on criteria for those specific categories. Blanket instructions ("confirmed only") are too vague to improve precision.

PRACTICE TAKEAWAY

Guiding principle: *High false-positive categories erode trust in the accurate ones; disable the noisy categories while you fix them.*
Design guidance: *Temporarily disable high-noise finding categories to restore trust, then refine their criteria before re-enabling.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q45 D5 Claude Code for CI/CD CORRECT

Your CI pipeline uses Claude to generate unit tests for a new authentication module, then immediately asks the same Claude session to review the generated tests for coverage gaps. QA analysis of 40 generated test suites finds that the self-review step flags gaps in only 8% of suites, but manual inspection reveals meaningful coverage gaps in 61% of them. What is the most accurate diagnosis and fix?

A The review prompt is too vague — add explicit coverage criteria specifying which edge cases must be validated for authentication code.

B The generated tests are passed to the review step without the original requirements, so the reviewer cannot check whether all requirements are covered.

C The same session that generated the tests retains the generation reasoning context, making it less likely to question its own assumptions — use a separate Claude instance with only the tests and requirements as input.

D The 8% flagging rate indicates the tests are genuinely high quality — the 61% gap rate from manual inspection reflects subjective differences between human reviewers and the model.

WHY THE OTHER CHOICES MISS

OPTION A

Explicit criteria improve review quality but do not address the root cause: the same session's generation context suppresses critical questioning regardless of how specific the review prompt is.

OPTION B

Missing requirements context would cause the reviewer to miss requirement-specific gaps, but the 61% gap rate includes structural gaps visible from the tests alone — context isolation from generation is the primary cause.

CORRECT RATIONALE (OPTION C)

Task Statement 4.6 documents the self-review limitation: a model retains the reasoning context from generation, which suppresses questioning of its own assumptions. An independent Claude instance starting from only the tests and requirements — with no generation context — is significantly more effective at catching coverage gaps, approaching the review without the assumptions that shaped the original output.

OPTION D

A 61% gap rate confirmed across 40 test suites by multiple reviewers is not subjective variance — it is a systematic pattern attributable to self-review bias from retained generation context.

PRACTICE TAKEAWAY

Guiding principle: *A model that generated code retains the reasoning context that shaped it, systematically reducing its ability to question its own assumptions during review.*

Design guidance: *Use a separate Claude instance with only the deliverable and requirements as input for review — no generation context — when review quality is critical.*

Review topic: Context Management

Q46 D5 Code Generation with Claude Code CORRECT

A developer is 90 minutes into a large refactoring session that has gone through three `/compact` compressions. After the third compression, the agent begins recommending an approach that contradicts a specific architectural constraint it correctly referenced 45 minutes earlier — avoiding synchronous calls in the event handler layer. The constraint was explicitly discussed and agreed upon early in the session. What is the most likely cause?

A The `/compact` algorithm randomly reorders context items during compression, moving older constraints after less relevant recent details.

B Progressive `/compact` summarization condensed the specific constraint and its rationale into a vague summary that no longer communicates the restriction precisely.

C Context compression causes early tool results to expire, so the agent can no longer access the original architectural decision.

D The model version changed between compression cycles, which caused the agent to reinterpret the event-handler constraint differently.

WHY THE OTHER CHOICES MISS

OPTION A

Context compression does not randomly reorder content; the issue is precision loss during summarisation, not ordering errors.

CORRECT RATIONALE (OPTION B)

Task Statement 5.1 documents progressive summarisation risks: condensing specific decisions, constraints, and rationale into vague summaries loses the precision needed to maintain consistency. Key facts should be maintained outside the summarised history in structured blocks.

OPTION C

`--resume` restores the session but does not prevent summarisation; the underlying issue requires keeping critical decisions in persistent structured notes.

OPTION D

Model versions do not change between compression cycles within a session.

PRACTICE TAKEAWAY

Guiding principle: *Context compression preserves continuity, but it can blur exact constraints unless they are maintained as structured facts.*
Design guidance: *Keep architectural decisions, invariants, and rationale in a durable facts block or scratchpad before relying on `/compact` during long work.*

Review topic: Context Management

Q47 D4 Code Generation with Claude Code INCORRECT

Claude is implementing an adapter that converts partner webhook payloads into your internal event format. After several attempts, the code still places customer

fields under the wrong parent object and preserves partner-local timestamps instead of normalized UTC strings. Your instructions have described the mapping in prose, but each revision fixes one case while breaking another. What should you provide next?

A A longer prose specification that lists every field path, null-handling rule, and timestamp format in a single requirements section.

B Two or three representative partner payloads paired with the exact internal event objects they should produce, including nested fields and timestamp normalization.

C A request for Claude to restate its current understanding of the adapter contract before making another code change.

D A JSON schema for the internal event object and an instruction to validate every adapter result against that schema.

WHY THE OTHER CHOICES MISS

OPTION A

More prose can still be interpreted differently, especially when multiple nested fields and edge cases interact.

CORRECT RATIONALE (OPTION B)

Concrete input-output pairs remove the ambiguity that remains in prose mapping descriptions. They show the exact nesting, field moves, omitted values, and timestamp conversion the implementation must reproduce across representative cases.

OPTION C

Asking Claude to explain its interpretation may reveal the misunderstanding, but it still does not provide unambiguous target outputs for the next implementation.

OPTION D

A schema can enforce required fields and types, but it usually cannot prove that each partner field was mapped to the intended destination or that example-specific timestamp conversions are correct.

PRACTICE TAKEAWAY

Guiding principle: *For ambiguous transformations, examples communicate the target behavior more precisely than another paragraph of requirements.*
Design guidance: *Provide 2-3 representative inputs paired with exact expected outputs, covering nested fields, normalization, nulls, and edge cases.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q48 D1 Code Generation with Claude Code CORRECT

Your code generation system lets developers submit feature requests via a web form. A coordinator spawns a requirements clarification subagent, a code generation subagent, and a test generation subagent. Developer satisfaction surveys show a 62% "too much friction" rating, primarily because the requirements subagent asks an average of 12 clarifying questions per request — including database schema questions for a simple button-label change. What should change?

A Remove the requirements clarification subagent entirely and require developers to submit complete specifications in their initial feature request forms.

B Reduce the clarification subagent's `max_tokens` budget to limit the number of questions it can generate in a single response.

C Classify the feature type first.

D Set a global maximum of 3 clarification questions per request in the coordinator's system prompt regardless of the feature type or complexity.

WHY THE OTHER CHOICES MISS

OPTION A

Removing clarification entirely leads to incomplete specs and poor generation quality for ambiguous requests.

OPTION B

max_tokens limits output length but causes truncated questions, not fewer targeted ones.

CORRECT RATIONALE (OPTION C)

The issue is that one-size-fits-all clarification produces unnecessary questions for common feature types. Categorizing first and using type-specific templates ensures the depth of clarification matches feature complexity — reducing friction for simple cases while maintaining rigor for complex ones.

OPTION D

A global 3-question limit causes the same problem for complex features where 12 questions are genuinely needed.

PRACTICE TAKEAWAY

Guiding principle: *Coordinators should route by request type and complexity, not apply one fixed flow to every request.*
Design guidance: *Classify the request first and select a type-appropriate depth of clarification or subagent path.*

Review topic: Agentic Architecture

Q49 D3 Code Generation with Claude Code CORRECT

Your project uses co-located test files — `Button.test.tsx` lives next to `Button.tsx`, `api.test.ts` next to `api.ts`, and so on across 40 directories. You want Claude to apply testing-specific conventions (mock patterns, assertion style, coverage thresholds) only when editing test files, regardless of which directory they live in. What is the correct approach?

A Create a file in `.claude/rules/` with YAML frontmatter `paths: ["**/*.test.tsx", "**/*.test.ts"]` so testing conventions activate only when Claude edits files matching those glob patterns.

B Place a `CLAUDE.md` file in each of the 40 directories that contains test files, repeating the testing conventions in each location so they are always present when editing tests.

C Add a `[Testing Conventions]` section to the root `CLAUDE.md` and rely on Claude to apply the section contextually when it detects it is editing a test file.

D Create a `/testing-conventions` slash command that developers invoke manually each time before editing a test file to load the testing-specific standards.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Task Statement 3.3 specifies `.claude/rules/` files with YAML frontmatter `paths` fields for conditional rule activation. Path-scoped rules load only when editing matching files, regardless of directory location — exactly what is needed for test files spread throughout the codebase.

OPTION B

Placing CLAUDE.md in every test-containing directory requires duplicating conventions and misses tests in directories without one.

OPTION C

OPTION D

Section headers do not create conditional loading; the entire CLAUDE.md is always loaded.

Skills require manual invocation and are not automatically triggered by file type.

PRACTICE TAKEAWAY

Guiding principle: *Use deterministic context loading when conventions depend on file path or file type.*
Design guidance: *Put cross-cutting conventions in* `.claude/rules/` *with glob frontmatter instead of duplicating CLAUDE.md files or relying on manual commands.*

Review topic: Claude Code Config

Q50 D1 Code Generation with Claude Code INCORRECT

Your code generation pipeline has a coordinator that spawns a `generate_code` subagent and a `write_tests` subagent in sequence for each feature request. After running on 30 features, developers report that the generated tests consistently validate the behavior described in the spec rather than the actual behavior of the generated code — catching spec interpretation differences but missing implementation bugs. What is the root cause?

A The `write_tests` subagent uses a different model version than `generate_code`, causing edge-case interpretation differences between the two subagents working from the same specification.

B The coordinator instructs both subagents to follow the specification independently, so each interprets edge cases differently without having access to the other subagent's output.

C The `write_tests` subagent's `allowedTools` does not include a Read tool, so it cannot access the generated code files written by the `generate_code` subagent to disk.

D The `write_tests` subagent is never given the generated code.

WHY THE OTHER CHOICES MISS

OPTION A

Model version differences don't cause tests to validate spec rather than code — the issue is that the test subagent never saw the code.

OPTION B

Following the spec independently is the symptom, not the cause — the cause is missing code context in the test subagent's prompt.

OPTION C

Read tool availability affects whether the subagent can read files it knows about, but the subagent wasn't given the generated code's location either.

CORRECT RATIONALE (OPTION D)

Task Statement 1.3 specifies that subagents do not automatically share context or outputs — the coordinator must explicitly include the generated code in the `write_tests` subagent's prompt for tests to validate actual implementation behavior.

PRACTICE TAKEAWAY

Guiding principle: *Subagents do not inherit each other's context; the coordinator must pass every needed input explicitly.*
Design guidance: *Collect all subagent outputs and forward the required findings, file paths, source metadata, and generated artifacts into downstream prompts.*

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q51 D1 Code Generation with Claude Code INCORRECT

Your code generation agent is tasked with generating a feature, running tests, and looping until all tests pass or a maximum of 10 iterations is reached. After generating an initial implementation, the agent sees 3 failing tests. On the next iteration it generates completely new code from scratch rather than targeting the specific 3 failures. This pattern repeats across all 10 iterations without convergence, wasting the full API budget on each run. What is the root cause?

A The `run_tests` tool output is too verbose.

B The coordinator does not pass the prior iteration's code and specific failure messages into the next iteration's prompt.

C The agent's `allowedTools` does not include a Read tool, preventing it from reading the previously generated code files to understand what specific logic needs to change.

D The 10-iteration maximum is too low.

WHY THE OTHER CHOICES MISS

OPTION A

CORRECT RATIONALE (OPTION B)

Verbose test output is a contributing factor but not the root cause — even with clear output, the agent can't fix what it doesn't carry forward.

Without the previous iteration's code and specific failure messages in the next prompt, the agent treats each iteration as a fresh start. The coordinator must pass both the prior code and the specific failing test names/messages to enable targeted fixes rather than full regeneration.

OPTION C

The agent can generate new code without reading prior code — but it should be reading and fixing, which requires context carryover.

OPTION D

More iterations of random regeneration don't converge — targeted fixes based on failure context are required.

PRACTICE TAKEAWAY

Guiding principle: *Subagents do not inherit each other's context; the coordinator must pass every needed input explicitly.*
Design guidance: *Collect all subagent outputs and forward the required findings, file paths, source metadata, and generated artifacts into downstream prompts.*

Review topic: Agentic Architecture

Review D1 in Cheat Sheet

Q52 D4 Code Generation with Claude Code INCORRECT

A code generation agent produces JSDoc comments for all generated functions. Analysis of 200 generated functions shows 29% are missing `@param` tags for one or more parameters and 17% omit `@returns` entirely. The system prompt includes a detailed description of the required JSDoc format. This prose specification has not resolved the inconsistency across 6 weeks of production use. What is the most reliable fix?

A Add 5 few-shot examples of correctly formatted JSDoc comments for different function signatures.

B Add a post-generation lint step using ESLint with the `valid-jsdoc` rule to detect and flag non-compliant output, triggering regeneration for functions with missing tags.

C Define the JSDoc standard in CLAUDE.md so the requirement persists as a project-level standard across all code generation sessions without needing to be repeated in each

system prompt.

D Define a `generate_jsdoc` tool with a JSON schema that requires a `params` array (one entry per parameter with `name` and `description`) and a non-nullable `returns` field.

WHY THE OTHER CHOICES MISS

OPTION A

Few-shot examples are effective for consistent formatting (TS 4.2) but are probabilistic — the model can still omit tags on novel function signatures not covered by the examples. The question asks for the most reliable fix, and schema enforcement is stronger than example-based anchoring.

OPTION B

Post-generation linting catches non-compliance after the fact and triggers regeneration — it adds latency and retry overhead rather than preventing the structural omission at generation time.

OPTION C

Defining the standard in CLAUDE.md makes the same prose description persistent — it does not change the enforcement mechanism and will produce the same inconsistency from a different location.

CORRECT RATIONALE (OPTION D)

Task Statement 4.3 identifies JSON schema via `tool_use` as "the most reliable approach for guaranteed schema-compliant output." A required `params` array and non-nullable `returns` field make omission a structural failure — the agent cannot produce a schema-valid response without including all required tags. Prose instructions describe the format; a schema enforces it.

PRACTICE TAKEAWAY

Guiding principle: *When output shape must be guaranteed, schema enforcement is stronger than prose, examples, or post-hoc linting.*

Design guidance: *Use tool-based structured output for required fields and non-nullable values, then reserve examples for semantic nuance the schema cannot express.*

Review topic: Prompt Engineering

Review D4 in Cheat Sheet

Q53 D4 Code Generation with Claude Code CORRECT

Your code-generation pipeline includes a self-review step that comments on generated diffs. Developers complain it flags too much: cosmetic style nits, alphabetical-ordering opinions, and "consider renaming" suggestions drown out the occasional real bug. Telling it to "only report important issues" changed nothing. What is the most effective fix?

A Add an instruction to "be conservative and only surface high-confidence findings" so the model self-limits what it reports.

B Define explicit categories to report (correctness bugs, security issues, broken contracts) and explicit categories to skip (style, naming, local-pattern preferences).

C Lower the model temperature so the review produces fewer, more deterministic comments per diff.

D Cap the review at three comments per file so the noise is bounded regardless of what it finds.

WHY THE OTHER CHOICES MISS

OPTION A

"Be conservative / high-confidence" is the same vague guidance that already failed; it does not define what counts as worth reporting.

CORRECT RATIONALE (OPTION B)

Vague qualifiers like "important" or "conservative" do not improve precision (TS 4.1). Defining explicit report/skip categories gives the model concrete, testable criteria, cutting the false positives that erode trust without suppressing real bugs.

OPTION C

Temperature changes phrasing variability, not the model's judgment about which categories of issue matter.

OPTION D

A fixed cap can drop a real bug to stay under the limit while still spending the budget on nits — it bounds volume, not relevance.

PRACTICE TAKEAWAY

Guiding principle: *Explicit report/skip criteria cut false positives better than vague "be conservative" instructions.*
Design guidance: *Name the categories to report and the categories to skip so the model has a testable filter.*

Review topic: Prompt Engineering

Q54 D2 Code Generation with Claude Code INCORRECT

Your team of six developers wants to use a GitHub MCP server for PR lookups and CI status checks during code generation workflows. Each developer has their own personal access token. You want consistent tooling across the team without committing any credentials to the repository. What is the correct configuration approach?

A Add the server to project-scoped `.mcp.json` with `${GITHUB_TOKEN}` environment variable expansion for authentication, and document the required variable in the project README.

B Have each developer add the GitHub MCP server to their personal `~/.claude.json` with their own token so credentials never touch the project repository.

C Create a `.env` file with a shared service account token and reference it from `.mcp.json`, then add `.env` to `.gitignore` to prevent accidental commits.

D Configure the server in project-scoped `.mcp.json` with a placeholder token value, then instruct developers to override it locally.

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

Task Statement 2.4 documents project-scoped `.mcp.json` with environment variable expansion (e.g., `${GITHUB_TOKEN}`) as the correct pattern for shared team MCP servers: a single version-controlled configuration that each developer populates with their own credential in their local environment. Documenting the required variable enables easy onboarding without committing any secret.

OPTION B

User-scoped configuration in `~/.claude.json` keeps credentials personal but means the GitHub tools are not automatically available to developers who clone the repo — each must manually configure the server.

OPTION C

A shared service account token in `.env` is a security risk, and `.gitignore` approaches are fragile — `${GITHUB_TOKEN}` expansion in `.mcp.json` solves this without any shared secret.

OPTION D

A placeholder token requires each developer to know the override syntax and creates an inconsistent configuration state that is difficult to debug and onboard from.

PRACTICE TAKEAWAY

Guiding principle: *Team MCP servers belong in project-scoped* `.mcp.json` *with environment variable expansion so configuration is shared without exposing credentials.*

Design guidance: *Use* `${ENV_VAR}` *syntax in* `.mcp.json` *for authentication tokens, and document required environment variables so developers can onboard without configuration guesswork.*

Review topic: Tool Design & MCP

Review D2 in Cheat Sheet

Q55 D4 Code Generation with Claude Code CORRECT

You are adding cache invalidation to a payment processing service. The codebase uses three different caching layers with different invalidation APIs, and the business rules for when to invalidate after a refund are undocumented. You ask Claude Code to implement it. What approach best surfaces the design considerations you may not have anticipated before any code is written?

A Begin implementation and use failing tests as discovery — errors will reveal which cache layers and invalidation paths are relevant.

B Ask Claude to read all cache-related files first, then propose an implementation plan based on what it discovers.

C Ask Claude to interview you with questions about the caching strategy, consistency requirements, and failure tolerance before suggesting an implementation.

D Provide Claude with the three cache library docs and ask it to produce a comparison table before choosing an approach.

WHY THE OTHER CHOICES MISS

OPTION A

Failing tests reveal implementation bugs, not the upstream design decisions about which cache layers to invalidate and under what conditions.

OPTION B

File reading builds Claude's understanding of what exists, but cannot surface the business rules and consistency requirements that only the developer can provide.

CORRECT RATIONALE (OPTION C)

OPTION D

Task Statement 3.5 documents the interview pattern for implementation in unfamiliar domains: having Claude ask questions surfaces consistency requirements, failure modes, and business rules that only the developer knows — before implementation commits to an approach. Payment cache invalidation involves non-obvious tradeoffs (eventual vs. strong consistency, partial vs. full invalidation) that prose instructions alone cannot anticipate.

A library comparison answers the wrong question — which API to use — rather than surfacing the business rules and failure modes that should drive the design.

PRACTICE TAKEAWAY

Guiding principle: *The interview pattern surfaces implicit requirements before implementation, preventing costly rework from undiscovered constraints in unfamiliar domains.*

Design guidance: *Use the interview pattern when implementing features where business rules, consistency targets, or failure modes are undocumented and only the developer can provide them.*

Review topic: Prompt Engineering

Q56 D5 Code Generation with Claude Code CORRECT

Your code-generation agent works in phases: it first explores the target service (reading ~30 files), then implements a new endpoint. By the time it reaches implementation, its context is full of raw file dumps from exploration and it starts misremembering the interfaces it just read. What is the most effective way to structure this?

A Increase the context window tier so all 30 raw file dumps and the implementation work fit together without crowding.

B After the exploration phase, summarize the key findings (interfaces, conventions, file locations) into a compact brief and start the implementation phase from that summary instead of the raw dumps.

C Run exploration and implementation in one continuous pass so the agent never loses the thread between phases.

D Lower `max_tokens` during exploration so the file dumps take up less of the context window.

WHY THE OTHER CHOICES MISS

OPTION A

A bigger window postpones the problem and raises cost; the raw dumps still dilute attention on the interfaces that matter.

CORRECT RATIONALE (OPTION B)

Verbose exploration output accumulates and degrades attention by the implementation phase (TS 5.4). Summarizing the phase-1 findings into a compact brief and injecting that — rather than the raw file dumps — into the implementation phase keeps the context lean and the interfaces reliable.

OPTION C

A single continuous pass is exactly what is failing — the exploration output is what crowds out the interface details.

OPTION D

Lowering max_tokens shortens responses but does not summarize or remove the accumulated file dumps already in context.

PRACTICE TAKEAWAY

Guiding principle: *Summarize a phase's key findings before the next phase instead of carrying raw output forward.*
Design guidance: *Compress exploration output into a brief (interfaces, conventions, locations) and start the next phase from that summary.*

Review topic: Context Management

Q57 D1 Code Generation with Claude Code CORRECT

Your code generation pipeline builds features across three layers using specialized subagents: frontend components, backend APIs, and database schemas. After generating a complete feature, integration tests fail because the frontend is calling `/api/v2/users` but the backend only exposes `/api/v1/users`, and the database schema uses `user_id` while the backend ORM expects `userId`. What is the correct approach to catch these mismatches before integration tests run?

A Have the coordinator collect all three subagent outputs and pass them to a dedicated validation subagent whose explicit task is checking cross-layer consistency.

B Have each subagent read the other subagents' generated files directly before returning its final output to the coordinator.

C Enable extended thinking on the coordinator so it can reason through cross-layer consistency across all three outputs without needing a separate validation subagent.

D Re-run each subagent with an added instruction: "Verify your output is consistent with the other layers"

WHY THE OTHER CHOICES MISS

CORRECT RATIONALE (OPTION A)

A dedicated validation subagent receiving all three outputs explicitly is the correct multi-agent pattern for cross-layer consistency checking. The coordinator aggregates outputs and provides them as context — subagents don't share context automatically.

OPTION B

Subagents don't automatically know what files other subagents produced — the coordinator must provide this information explicitly.

OPTION C

Extended thinking helps reason about complex single-agent problems; cross-layer consistency with outputs scattered across subagents requires explicit aggregation.

OPTION D

Re-running each subagent with a vague consistency instruction doesn't work — they still lack each other's actual output to compare against.

PRACTICE TAKEAWAY

Guiding principle: *Subagents do not inherit each other's context; the coordinator must pass every needed input explicitly.*
Design guidance: *Collect all subagent outputs and forward the required findings, file paths, source metadata, and generated artifacts into downstream prompts.*

Review topic: Agentic Architecture

Q58 D3 Code Generation with Claude Code CORRECT

A lead developer added TypeScript coding standards to `~/.claude/CLAUDE.md` on their own machine. When three other team members run the same project, they

report the agent applies no coding standards at all. All four developers are running the same Claude Code version from the project root. What is the root cause?

A The `~/.claude/CLAUDE.md` file requires a YAML frontmatter activation header.

B User-level `~/.claude/CLAUDE.md` is not in version control.

C User-level CLAUDE.md files require explicit opt-in.

D The `~/.claude/` directory is only read on macOS.

WHY THE OTHER CHOICES MISS

OPTION A

No activation header or frontmatter is required for CLAUDE.md files.

CORRECT RATIONALE (OPTION B)

Task Statement 3.1 states that user-level settings in `~/.claude/CLAUDE.md` apply only to that user and are not shared via version control. Team-wide standards must be in the project-level `CLAUDE.md` so they are distributed when developers clone or pull.

OPTION C

No opt-in command exists; CLAUDE.md files are loaded automatically.

OPTION D

The `~/.claude/` directory is supported across platforms.

PRACTICE TAKEAWAY

Guiding principle: *Behavior depends on where config is scoped; user-level CLAUDE.md is not shared via version control.*
Design guidance: *Put team-wide standards in the project CLAUDE.md so every clone receives them.*

Review topic: Claude Code Config

Q59 D5 Code Generation with Claude Code CORRECT

You are about to remove a deprecated `processLegacyOrder()` function. Before removing it you need to discover all 47 files that call it across a 200,000-line codebase — a task requiring extensive Grep searches, file reads, and import

graph traversal. What is the primary reason to delegate this discovery phase to the Explore subagent rather than running it in the main session?

A The Explore subagent has access to a larger context window than the main session, allowing it to read more files in a single pass without needing to paginate through results.

B Subagents run in parallel background threads, so the 47-file discovery completes faster than it would if run as sequential tool calls in the main session.

C The Explore subagent has elevated filesystem permissions that allow it to read protected files and build import graphs that the main session's user account cannot access.

D Delegating verbose discovery to a subagent keeps the main session context clean.

WHY THE OTHER CHOICES MISS

OPTION A

Subagents share the same context window constraints as the main session.

OPTION B

Subagents do not automatically parallelise; parallelism requires explicitly spawning multiple concurrent subagents.

OPTION C

Subagents operate under the same user permissions as the main session.

CORRECT RATIONALE (OPTION D)

Task Statement 5.4 documents delegating verbose discovery phases to subagents to prevent context window exhaustion during multi-phase tasks. Running 47 file reads and Grep searches in the main session fills the context with verbose intermediate output, degrading the implementation phase that follows. The subagent returns a structured summary to the main session, keeping the main context clean for subsequent work.

PRACTICE TAKEAWAY

Guiding principle: *Noisy discovery should not consume the same context you need for design and implementation.*

Design guidance: *Use an Explore subagent for broad searches and file reads, then bring back a concise, structured summary for the main conversation.*

Review topic: Context Management

Q60 D3 Code Generation with Claude Code INCORRECT

Your team created an `/analyze-codebase` skill that performs comprehensive dependency scanning, test coverage calculation, and code quality metrics. After running this skill, team members report that Claude loses track of their original task — the session fills with verbose discovery output and Claude begins treating the analysis as the primary goal. What skill frontmatter option most directly addresses this?

A Add `model: haiku` to the skill's frontmatter to use a faster, less verbose model for analysis output.

B Add `context: fork` to run the skill in an isolated sub-agent context, preventing verbose output from accumulating in the main conversation.

C Split the skill into three smaller skills — dependency, coverage, and quality — so each produces less output per invocation.

D Add instructions to the skill to compress all outputs into a brief summary before displaying results.

WHY THE OTHER CHOICES MISS

OPTION A

A smaller model produces less verbose responses but still returns output to the main context — it does not provide isolation from the main conversation.

CORRECT RATIONALE (OPTION B)

`context: fork` in skill frontmatter runs the skill in an isolated sub-agent context. Verbose discovery output stays confined to that sub-agent; only a concise summary returns to the main conversation. This preserves the developer's original task context while delivering the full analysis capability.

OPTION C

Splitting into three skills reduces output per invocation but each invocation still accumulates in the main context — the root cause is context isolation, not output volume per skill.

OPTION D

Summary compression instructions are probabilistic and rely on the model's judgement about what to compress — `context: fork` provides deterministic isolation.

PRACTICE TAKEAWAY

Guiding principle: *Skills that generate verbose exploratory output should run in forked sub-agent contexts to protect the main session's task focus.*

Design guidance: *Add* `context: fork` *to skill frontmatter when the skill's output would pollute the main conversation if returned verbatim.*

Review topic: Claude Code Config

Review D3 in Cheat Sheet

Back to results | New exam | Dashboard